

Information Retrieval: Assignment 1

May 13, 2026

1 Introduction

The main goal of this project was to classify a writer's l1, or first language(s), based on text written in Swedish. The reason this task was selected was, in part, due to the convenience in relation to the dataset selected, but also due to the possible real-world applications such a classifier could provide. For one, being able to identify information about a speaker can be very useful in a field like forensic linguistics, such as identifying key information about hackers based on demand letters, as has been a relevant issue of central discussion for Canvas. A classifier like this could also be used in learning: for example, for an education service teaching a language like Swedish, being able to identify a speaker's native language can allow a system to offer specialised support for issues that might be more common for different l1 speakers, as their dominant language affect their familiarity with different parts of the language they intend to learn. A person who speaks German may need help with grammar but less with word construction, while a person who speaks Mandarin may first need help with vocabulary.

2 Methods and Materials

For the dataset of this assignment, a combined method of both natural data and synthetic data was used. The custom dataset was based on a subset of the Superlim dataset from AI Sweden¹, which included a Swedish sentence with a spelling error, as well as a column of indices for where the error occurred, a corrected sentence, and the writer's l1, level of Swedish, and more. The data contained over 3000 Swedish sentences. From this dataset, only the original sentences and corresponding writer's l1 was used. To supplement the data, 100 synthetic sentences were generated using Codex 5.5 with medium intelligence to represent native Swedish writers, as no Swedish writers were in the original dataset. Due to the fact that many of the entries for l1 included multiple languages listed by the writer, the task became multilabel rather than single label. To achieve multilabel prediction, the labels were pre-processed into separate languages, rather than a string of all languages written by the writer.

The model used was generated using Codex and used BERT-base-multilingual-cased to embed the sentence vectors, with a linear classifier layer, and BCEWithLogitsLoss as the loss metric. After training, the python script would generate a file of the model's metrics, which included accuracy, and individual accuracy, precision, recall, and f1 scores for each label in the data, as well as the label's support, or frequency of the label in the test data. After initial training, the hyper parameter's for the prediction threshold were changed and the number of epochs increased, as the first model has very poor results, having an initial accuracy of 0%, likely due to the prediction threshold being too high, and the number of epochs not being enough for the model to make generalisations across the data. The numbers were arbitrarily changed to decrease the threshold from 0,5 to 0,2 and increase the epochs from 3 to 10 but provided an inarguable improvement of results over the first iteration of the model.

3 Results

The accuracy for the model during training was 17,1%, while the model scored on an outstanding test dataset reached 11,94%. Many of the individual label metrics had recall, precision, and f1 scores of 0, and several labels had 0 support in the test data. In 85 labels and label combinations, 2 had non-zero scores, as can be found in the table below.

¹<https://huggingface.co/datasets/AI-Sweden/SuperLim>

	Precision	Recall	F1-score	support
Arabiska	0.23557692307692307	0.6363636363636364	0.34385964912280703	77
Persiska	0.21875	0.21875	0.21875	32

Table 1: Scores from two labels in the model evaluation

4 Discussion

Overall, it is clear that the model has a very low accuracy. It is likely the model’s bad results are in large part due to the dataset. There is such a variety in the training data that a majority of the labels simply aren’t represented in the test data, and have very infrequent representation in the training data. This affected the already low scores to be even worse. Future attempts may try to further supplement the data with synthetic data, possibly based on what is already available. However, the natural data should still be the majority, as Swedish is not a high resource language, and thus representations from synthetic data may not be as accurate as one may prefer when training a new model to be applicable to real, natural, data. It may also be worth only testing on natural data, though this was not possible in this assignment due to the fact that Swedish was completely missing in all the data.

Another significant problem of the dataset was the over-representation of the label “Arabiska”, which made up around a third of the dataset. This is clearly seen to affect the model, as the demo always predicts this tag above all other tags. In essence, the model, rather than learning to find patterns across the data to divide and categorise the data, optimised to predict all data as “Arabiska.” This gave the model overall a very low accuracy, with the lone tag having much higher accuracy. In table 1, this label has a much higher recall than precision, meaning the model scores many true positives correctly, but at the cost of prediction many false positives. Despite this, the label “Persiska” has a balanced recall and precision, though with lower scores than the “Arabiska” label. The support is also above the average for the labels, suggesting the labels has better support from the training data as well, which helped the model to predict the label without over-powering it.

5 Assignment evaluation

For this assignment, I used AI for generating the code of the model, generating the synthetic data, and to upload the model to huggingface. AI was also used to configure the app.py code from the example demo to my model. Uploading the dataset, minorly configuring the model’s hyperparameters, and coming up with ideas for the assignment was all done by me. Further, this report, even after correcting formatting, is written purely by me, a human, using only Word’s built-in spell check.

The AI tool used for this assignment was Codex, 5.5, with medium “intelligence”. This assignment was my first time properly using AI, as I feel using AI can easily trick one into believing they have learned and understood what the tool is doing, without knowing how to do it. It was somewhat enlightening to get hands-on experience with how a person may use AI to generate code, and how to work with this tool, especially in “steering” the LLM after the initial prompt. Overall, it is clear that AI can be almost annoyingly useful, especially for generating code. At times, it surprised me in catching things I did not mention in prompts, such as catching and “repairing” my previous attempt to convert a .py file into .ipynb to import the model into Google Colab when asking it to convert the model file. Although the fact that it “behaved” very human-like was somewhat unsettling, it did work well as a tool. That being said, I do not believe I will use it as much for future assignments as I have for this one. By having codex generate all code and orchestrating the upload, I feel I lost out on good experience with the model pipeline and architecture. Although it would not have been feasible to do this assignment without AI, I still feel limiting or completely excluding the use of AI is better in a learning environment, at least for myself.

6 Links

https://huggingface.co/uhygutfr/Swedish_BERT-based_L1_classifier
https://huggingface.co/datasets/uhygutfr/Swedish_sentences_with_l1
https://huggingface.co/spaces/uhygutfr/Swedish_BERT-based_L1_classifier_demo